УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Хеш-таблицы словари и множества

ЛЕКЦИЯ 8

Ахметов Ильнур Вазирович

План лекции

1. Понятие хеширования и оценка сложности операций (O-большое).
2. Словари (dict): создание, методы работы с ключами и значениями.
3. Множества (set): операции, обеспечение уникальности элементов.
4. Практическое применение хеш-таблиц для оптимизации алгоритмов.
5. Задачи аналитика: справочники, сводки, дубликаты, валидация.
6. Ловушки: изменяемые ключи, порядок, выбор структуры.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы уже умеем

- Лекция 7: поиск, сортировка, map/filter, агрегация.
- Линейный поиск — O(n), бинарный — O(log n).
- Но бинарный требует отсортированных данных.
- Главный вопрос этой лекции: можно ли найти
- элемент за ОДНО действие — O(1)?
- Ответ: да, если данные организованы как хеш-таблица.
- Так устроены словари и множества в Python.

консоль Python

```
>>> wells = {'K-12': 85.4, 'K-13': 12.7}
>>> wells['K-13']     # один шаг
12.7
>>> 'K-99' in wells
False

# 1 000 000 ключей -> всё равно 1 шаг
```

Понятие хеширования

- Хеш-функция превращает ключ в число-индекс.
- По индексу значение лежит прямо в массиве —
- доступ за одно действие, O(1).
- Хеш-таблица = массив + хеш-функция.
- Ключ -> hash(ключ) -> индекс корзины -> значение.
- Так работают dict и set в Python.
- Аналогия: гардероб — номерок определяет крючок.

```
схема
ключ 'K-13'
     |
hash('K-13') -> 48213
     |
48213 % 8 = 5   (индекс корзины)
     |
[0][1][2][3][4][5][6][7]
                ^
           значение 12.7
```

Функция hash() и её требования

- hash(x) — встроенная функция, возвращает целое число.
- Требование 1: стабильность — одинаковые ключи
- дают одинаковый хеш (в рамках одного запуска).
- Требование 2: равномерность — значения распределяются
- по корзинам без «скоплений».
- Требование 3: ключ должен быть НЕИЗМЕНЯЕМЫМ
- (числа, строки, кортежи).

консоль Python

```
>>> hash(42)
42
>>> hash('K-13')
-48213780581123
>>> hash('K-13') == hash('K-13')
True
>>> hash((1, 2))      # кортеж - можно
3713081631934410656
>>> hash([1, 2])      # список - нельзя
TypeError: unhashable type: 'list'
```

Коллизии: два ключа — одна корзина

- Коллизия — разные ключи попали в одну корзину.
- Это нормально: корзин меньше, чем возможных ключей.
- Python решает коллизии цепочкой: в корзине хранится
- список пар (ключ, значение).
- При поиске: вычислить хеш -> найти корзину ->
- перебрать пары в ней и сравнить ключи ==.
- Хорошая хеш-функция делает коллизии редкими.

```
схема коллизии
'K-13' -> корзина 5
'K-29' -> корзина 5   (коллизия!)

корзина 5: [('K-13', 12.7), ('K-29', 40.0)]

поиск 'K-29':
  хеш -> корзина 5
  'K-13' == 'K-29'? нет
  'K-29' == 'K-29'? да -> 40.0
```

Почему O(1) и когда становится O(n)

- Средний случай: одна корзина — мало пар -> O(1).
- Говорят «амортизированное O(1)» — в среднем по сериям.
- Худший случай: все ключи в одной корзине -> O(n).
- Это почти невозможно при хорошей хеш-функции.
- При заполнении таблица РЕХЕШИРУЕТСЯ: растёт,
- все ключи пересчитываются — единичная дорогая вставка.
- Итого: поиск/вставка/удаление — O(1) в среднем.

```
шкала
list.index(x)      O(n)      линейно
list (отсорт.)     O(log n)  бинарно
dict[key]          O(1)      хеш
x in set           O(1)      хеш
x in list          O(n)      линейно

# 1 млн элементов:
# list:  до секунды
# dict:  микросекунды
```

Сложность операций: шпаргалка

Проверка in: list O(n) — dict O(1) — set O(1)

Доступ по элементу: list[i] O(1) — dict[k] O(1)

Поиск значения: list.index O(n) — нет у dict по значению

Вставка: list.append O(1) — dict[k]=v O(1)

Вставка в середину: list.insert O(n)

Удаление: list.remove O(n) — dict.pop O(1)

Сортировка: sorted O(n log n) везде

сводная таблица

операция	list	dict	set
x in коллекция	O(n)	O(1)	O(1)
взять по ключу	O(n)	O(1)	--
добавить	O(1)	O(1)	O(1)
удалить	O(n)	O(1)	O(1)
по порядку	да	да*	нет
дубликаты	да	нет	нет

* dict помнит порядок вставки (3.7+)

Создание словаря

- Словарь — пары «ключ: значение».

Литерал: {ключ: значение, ...}

Функция dict(): dict(a=1, b=2) или dict(pairs)

Пустой: {} — всегда словарь, не множество!

fromkeys: одинаковое значение для списка ключей.

- Ключи уникальны: повтор — последнее значение.
- Ключи неизменяемы: числа, строки, кортежи.

консоль Python

```
>>> wells = {'K-12': 85.4, 'K-13': 12.7}
>>> wells2 = dict(K_14=65.3)
>>> wells2
{'K_14': 65.3}
>>> dict([('K-15', 40.0), ('K-16', 0.0)])
{'K-15': 40.0, 'K-16': 0.0}
>>> dict.fromkeys(['a','b'], 0)
{'a': 0, 'b': 0}
>>> {'x': 1, 'x': 2}   # повтор ключа
{'x': 2}
```

Чтение: [] и get()

d[key] значение по ключу; нет ключа -> KeyError.

d.get(key) нет ключа -> None (без ошибки).

d.get(key, default) нет ключа -> значение по умолчанию.

- get — безопасное чтение, [] — строгое.
- Ошибку KeyError лучше предотвращать, не ловить.
- None опасен: он может «протечь» в расчёты.

консоль Python

```
>>> wells = {'K-12': 85.4, 'K-13': 12.7}
>>> wells['K-12']
85.4
>>> wells['K-99']
KeyError: 'K-99'
>>> wells.get('K-99')
(ничего — None)
>>> wells.get('K-99', 0.0)
0.0
>>> wells.get('K-12', 0.0)
85.4
```

Добавление и обновление

`d[key] = value` и добавляет, и обновляет — одна запись.

- Есть ключ — значение заменится.
- Нет ключа — пара добавится.
- Это проще, чем у списка: нет append/insert.
- Вставка O(1) — независимо от размера словаря.
- Словарь может расти динамически, как список.

консоль Python

```
>>> wells = {'K-12': 85.4}
>>> wells['K-13'] = 12.7    # добавили
>>> wells['K-12'] = 90.0    # обновили
>>> wells
{'K-12': 90.0, 'K-13': 12.7}

# рост в цикле O(1) на шаг:
prices = {}
for name in ['a', 'b', 'c']:
    prices[name] = 0.0
```

Удаление

del d[key] удаляет пару; нет ключа -> KeyError.

d.pop(key) удаляет и ВОЗВРАЩАЕТ значение.

d.pop(key, default) без ошибки, вернёт default.

d.popitem() удаляет последнюю добавленную пару.

d.clear() очищает словарь целиком.

- pop с default — самый безопасный вариант.

консоль Python

```
>>> wells = {'K-12': 85.4, 'K-13': 12.7}
>>> wells.pop('K-13')
12.7
>>> wells
{'K-12': 85.4}
>>> wells.pop('K-99', None)
(нет ошибки)
>>> del wells['K-12']
>>> wells
{}
```

update() и слияние словарей

`d1.update(d2)` вливает d2 в d1 на месте.

- Совпадающие ключи — значение из d2 перезапишет.

`d1 | d2` новый объединённый словарь (Python 3.9+).

`d1 |= d2` объединение на месте.

- Порядок при конфликте: правый словарь побеждает.
- Типовое применение: словарь настроек + словарь переопределений.

консоль Python

```
>>> base = {'нефть': 85.4, 'вода': 0.2}
>>> fix  = {'нефть': 90.0, 'газ': 3.1}
>>> base.update(fix)
>>> base
{'нефть': 90.0, 'вода': 0.2, 'газ': 3.1}

>>> {'a': 1} | {'a': 9, 'b': 2}
{'a': 9, 'b': 2}     # правый победил
```

keys(), values(), items()

d.keys() все ключи (можно проверить in).

d.values() все значения (sum, max — работают).

d.items() пары (ключ, значение) для перебора.

- Все три — «представления», живут вместе со словарём.
- Ключи — как множество: без дубликатов, O(1) проверка.
- items() — главный инструмент при переборе.

консоль Python

```
>>> wells = {'K-12': 85.4, 'K-13': 12.7}
>>> list(wells.keys())
['K-12', 'K-13']
>>> list(wells.values())
[85.4, 12.7]
>>> sum(wells.values())
98.1
>>> max(wells.values())
85.4
>>> list(wells.items())
[('K-12', 85.4), ('K-13', 12.7)]
```

Перебор словаря

- for key in d — перебор ключей (по умолчанию).
- for v in d.values() — значения.
- for k, v in d.items() — сразу обе части (лучший).
- Не изменяйте словарь во время перебора!
- Собирайте изменения в новый словарь.
- Порядок — порядок вставки (Python 3.7+).

код

```
wells = {'K-12': 85.4, 'K-13': 0.0,
         'K-14': 65.3}

for k in wells:
    print(k)                # ключи

for k, v in wells.items():
    if v == 0:
        print(k, 'остановлена')

# K-13 остановлена
```

in и setdefault

`key in d` проверка наличия КЛЮЧА за O(1).

`value in d.values()` поиск по значению — O(n)!

- Значения ищутся линейно — это не хеш-поиск.

`d.setdefault(k, default)` если нет — вставить и вернуть.

- setdefault удобен для группировки без defaultdict.
- Проверяйте ключи, не значения — помните о сложности.

консоль Python

```
>>> wells = {'К-12': 85.4, 'К-13': 12.7}
>>> 'К-13' in wells
True
>>> 85.4 in wells.values()   # O(n)
True
>>> d = {}
>>> d.setdefault('Куст 1', []).append(85.4)
>>> d.setdefault('Куст 1', []).append(65.3)
>>> d
{'Куст 1': [85.4, 65.3]}
```

Вложенные словари: записи

- Значением может быть другой словарь — «запись».
- Это способ хранить таблицу без pandas.
- Внешний ключ — идентификатор записи.
- Внутренние ключи — поля записи.
- Доступ: db['К-12']['дебит'] — O(1) на уровень.
- Так часто моделируют реестры и каталоги.

код

```
db = {
    'К-12': {'дебит': 85.4, 'W': 0.24,
             'куст': 'Куст 1'},
    'К-13': {'дебит': 12.7, 'W': 0.91,
             'куст': 'Куст 2'},
}
print(db['К-12']['дебит'])   # 85.4
for wid, rec in db.items():
    if rec['куст'] == 'Куст 1':
        print(wid, rec['дебит'])
```

Словарные включения

- {ключ: значение for x in последовательность}
- Так строят словари одним выражением.
- Можно фильтровать: ... if условие.
- Можно «перевернуть» словарь (значения -> ключи).
- Ключи должны быть неизменяемыми и уникальными.
- Читаемость: включение читается как формула задачи.

код

```
names = ['K-12', 'K-13', 'K-14']
zero = {n: 0.0 for n in names}
# {'K-12': 0.0, 'K-13': 0.0, 'K-14': 0.0}

debits = {'K-12': 85.4, 'K-13': 0.0}
active = {k: v for k, v in debits.items() if v >
0}
# {'K-12': 85.4}

# переворот (значения уникальны!):
inv = {v: k for k, v in debits.items()}
```

Сортировка словаря

- Словарь нельзя отсортировать «на месте».

`sorted(d.items(), key=...)` список пар в нужном порядке.

- Из списка пар легко собрать новый словарь dict(...).
- Сортировка по значениям — key=lambda kv: kv[1].
- По убыванию — reverse=True.
- Так делают рейтинги из справочников.

код

```
wells = {'K-12': 85.4, 'K-13': 12.7,
         'K-14': 65.3}

by_debit = sorted(wells.items(),
                  key=lambda kv: kv[1],
                  reverse=True)
print(by_debit)
# [('K-12',85.4),('K-14',65.3),('K-13',12.7)]

rating = dict(by_debit)   # снова словарь
```

Фильтрация и преобразование записей

- Типовой конвейер: отфильтровать -> преобразовать.
- Включение по items() делает оба шага сразу.
- Это аналог GROUP BY/HAVING из SQL на минималках.
- Пустой результат — нормальный исход, проверяйте.
- Один проход по записям — O(n) суммарно.
- Помните: изменение словаря в цикле — ловушка.

код

```
db = {'K-12': 85.4, 'K-13': 0.0,
      'K-14': 65.3, 'K-15': 0.0}

stopped = {k: v for k, v in db.items() if v == 0}
print(stopped)      # {'K-13': 0.0, 'K-15': 0.0}

koeff = {k: round(v / 100, 2) for k, v in
db.items()}
print(koeff)
# {'K-12': 0.85, 'K-13': 0.0, ...}
```

Создание множества

- Множество — набор УНИКАЛЬНЫХ элементов без порядка.

Литерал: {1, 2, 3}

Из списка: set([1, 2, 2, 3]) -> {1, 2, 3}

Пустое: set() — а {} это ПУСТОЙ СЛОВАРЬ!

- Элементы — только неизменяемые (хешируемые).
- Дубликаты при создании отбрасываются молча.

консоль Python

```
>>> tags = {'нефть', 'газ', 'нефть'}
>>> tags
{'нефть', 'газ'}
>>> set([1, 2, 2, 3, 3, 3])
{1, 2, 3}
>>> set('абракадабра')
{'а', 'б', 'р', 'к', 'д'}
>>> type({})
<class 'dict'>
>>> type(set())
<class 'set'>
```

Уникальность элементов

- Главное свойство: элемент либо есть, либо нет.
- Дубликат при добавлении просто игнорируется.
- Уникальность определяется равенством == и хешем.
- 1 и 1.0 — «один и тот же» элемент!
- True == 1 — тоже совпадут (осторожно с bool).
- Отсюда главный приём: set(список) — убрать дубликаты.

консоль Python

```
>>> ids = ['K-12', 'K-13', 'K-12', 'K-14']
>>> unique = set(ids)
>>> len(ids), len(unique)
(4, 3)

>>> s = {1, 1.0, True}
>>> s
{1}          # 1 == 1.0 == True

>>> 'a' in {'a', 'b'}
True
```

frozenset: неизменяемое множество

- frozenset — множество, которое нельзя менять.
- Зато его можно класть В множество и в ключ словаря.
- Обычный set — изменяемый и нехешируемый.
- Применение: константы-справочники, ключи группировок.
- API такой же: in, |, &, -, ^, len.
- Если не меняете множество — задумайтесь о frozenset.

консоль Python

```
>>> fs = frozenset(['нефть', 'газ'])
>>> 'газ' in fs
True
>>> fs.add('вода')
AttributeError: ... no attribute 'add'

>>> groups = {frozenset({1,2}): 'пара',
...           frozenset({3}): 'один'}
>>> groups[frozenset({2, 1})]
'пара'
```

Методы изменения

s.add(x) добавить один элемент.

s.remove(x) удалить; нет элемента -> KeyError.

s.discard(x) удалить; нет элемента — молча.

s.pop() удалить и вернуть произвольный элемент.

s.clear() очистить.

s.update(другая) добавить все элементы.

- discard безопаснее remove для «возможно нет».

консоль Python

```
>>> s = {'нефть', 'газ'}
>>> s.add('вода')
>>> s.discard('сера')     # нет - молча
>>> s.remove('газ')
>>> s
{'нефть', 'вода'}
>>> s.update(['песок', 'соль'])
>>> s.pop()               # произвольный
>>> len(s)
3
```

Операции: |, &, -, ^

A | B объединение — всё из обоих.

A & B пересечение — общие элементы.

A - B разность — есть в A, нет в B.

A ^ B симметрическая разность — только в одном.

- Порядок в разности важен: A - B != B - A.
- Результат — НОВОЕ множество, исходные не меняются.

консоль Python

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5}
>>> a | b
{1, 2, 3, 4, 5}
>>> a & b
{3, 4}
>>> a - b
{1, 2}
>>> b - a
{5}
>>> a ^ b
{1, 2, 5}
```

Методы операций и их формы

a.union(b) то же, что a | b.

a.intersection(b) то же, что a & b.

a.difference(b) то же, что a - b.

- Методы принимают ЛЮБУЮ коллекцию: a.union([3, 4]).
- Операторы | & требуют множество с обеих сторон.

a |= b / a &= b те же операции на месте.

- Читаемость: слова понятнее символов новичку.

код

```
a = {1, 2, 3}
print(a.union([3, 4, 5]))      # OK
# print(a | [3, 4, 5])         # TypeError!

print(a.intersection({2, 9}))
# {2}

seen = set()
seen |= {'K-12', 'K-13'}      # на месте
```

Подмножества и надмножества

`A <= B` A — подмножество B (все элементы A в B).

`A >= B` A — надмножество B.

`A < B` строгое подмножество (A != B).

- Проверка «все обязательные поля заполнены».
- Проверка «список тегов входит в допустимые».
- Это O(len(A)) — проход по меньшему множеству.

консоль Python

```
>>> need = {'дебит', 'давление'}
>>> have = {'дебит', 'давление', 'W'}
>>> need <= have
True
>>> have <= need
False
>>> {1, 2} < {1, 2}
False            # строгое подмножество
>>> {1} < {1, 2}
True
```

set против list: скорость in

- in по списку — линейный проход O(n).
- in по множеству — хеш-поиск O(1).
- Построить set — O(n) один раз.
- Если проверок много — set окупается мгновенно.
- Правило: frequent in -> set/dict, а не list.
- Это главная оптимизация из лекции 7 на практике.

сравнение

```
# 10 000 элементов, 10 000 проверок

list:  10 000 * 10 000 = 100 000 000
set:   10 000 (построение) + 10 000

# выигрыш ~5000 раз

ids_set = set(big_list)      # один раз
for x in requests:
    if x in ids_set: ...     # O(1)
```

Перебор, размер, сортировка

- Множество итерируется: for x in s.

len(s) — количество уникальных элементов.

- Порядок при переборе не гарантирован — не полагайтесь.

sorted(s) — отсортированный список элементов.

a.isdisjoint(b) True, если общих элементов нет.

- Пуст ли set: len(s) == 0 или not s.

консоль Python

```
>>> s = {'газ', 'нефть', 'вода'}
>>> len(s)
3
>>> sorted(s)
['вода', 'газ', 'нефть']
>>> {'сера'}.isdisjoint(s)
True
>>> {'газ'}.isdisjoint(s)
False
>>> not s        # пусто ли?
False
```

Оптимизация: set вместо list

- Паттерн: много проверок «а есть ли это?».
- Было: if x in big_list — O(n) на проверку.
- Стало: set один раз, проверки O(1).
- Сложность падает с O(n·m) до O(n + m).
- Это решает «висящие» скрипты на больших данных.
- Помните про память: set занимает больше места.

код

```
# было: O(n * m)
for req in requests:
    if req in all_ids:      # list!
        ...

# стало: O(n + m)
known = set(all_ids)        # один раз
for req in requests:
    if req in known:        # O(1)
        ...
```

Словарь как справочник

- Справочник: ключ — идентификатор, значение — данные.
- Замена цикла поиска на прямой доступ.
- Идеален для связи двух таблиц по ключу.
- Проверка наличия: if wid in catalog.
- Отсутствующие — обрабатывайте через get/in.
- Это главный инструмент соединения данных (join).

код

```
catalog = {'К-12': 'Куст 1',
           'К-13': 'Куст 2'}

for wid, debit in measurements:
    if wid in catalog:
        cluster = catalog[wid]   # O(1)
        print(wid, cluster, debit)
    else:
        print(wid, 'нет в справочнике!')
```

Поиск пары с заданной суммой

- Классика: найти в списке два числа с суммой S.
- Наивно: два цикла — O(n²).
- С множеством: один проход, ищем S - x в seen.
- Сложность O(n) вместо O(n²).
- На миллионе элементов: секунды против часов.
- Это стандартный приём на собеседованиях.

код

```
def find_pair(nums, target):
    seen = set()
    for x in nums:
        if target - x in seen:
            return (target - x, x)
        seen.add(x)
    return None

print(find_pair([85.4, 12.7, 52.0, 53.3], 66.0))
# (12.7, 53.3)   т.к. 12.7 + 53.3 = 66.0
```

Counter: подсчёт частот

- Counter — словарь «значение -> количество».
- Создаётся из любой последовательности.

most_common(n) — топ по частоте.

- Поддерживает арифметику: сложение, вычитание.
- Замена ручного цикла со счётчиком.
- Применение: статусы, категории, ошибки, теги.

консоль Python

```
>>> from collections import Counter
>>> st = ['ok','err','ok','warn','ok','err']
>>> c = Counter(st)
>>> c
Counter({'ok': 3, 'err': 2, 'warn': 1})
>>> c.most_common(1)
[('ok', 3)]
>>> Counter(st) - Counter(['ok'])
Counter({'ok': 2, 'err': 2, 'warn': 1})
```

defaultdict: группировка

- defaultdict(list) — автоматически создаёт список.
- Не нужно проверять «есть ли ключ».

groups[k].append(v) — одна строка вместо трёх.

- Фабрика может быть любой: list, int, set.
- После сбора — обычный словарь: перебор и агрегация.
- Это ручной GROUP BY, мостик к pandas.

код

```
from collections import defaultdict

groups = defaultdict(list)
for wid, (cluster, debit) in data:
    groups[cluster].append(debit)

avg = {c: sum(v)/len(v)
       for c, v in groups.items()}
print(avg)   # {'Куст 1': 75.35, ...}
```

Дедупликация с сохранением порядка

- set(...) убирает дубликаты, НО перемешивает порядок.
- Для отчётов порядок часто важен (по датам).

list(dict.fromkeys(seq)) — дедупликация с порядком.

- Работает с Python 3.7+: dict помнит вставку.
- Альтернатива: set + цикл с проверкой seen.
- Сложность всё равно O(n).

консоль Python

```
>>> events = ['K-12', 'K-13', 'K-12', 'K-14']
>>> set(events)
{'K-13', 'K-12', 'K-14'}   # порядок сломан

>>> list(dict.fromkeys(events))
['K-12', 'K-13', 'K-14']   # порядок сохранён
```

Кэш и мемоизация

- Мемоизация — кэш результатов по аргументам.
- Словарь: ключ — аргументы, значение — результат.
- Повторный вызов с теми же аргументами — O(1).
- Превращает экспоненциальную рекурсию в линейную.
- В стандартной библиотеке есть functools.lru_cache.
- Применение: дорогие расчёты, справочные запросы.

код

```
cache = {}
def slow_calc(key):
    if key in cache:          # O(1)
        return cache[key]
    result = expensive(key)   # долгий расчёт
    cache[key] = result
    return result

# либо готовое решение:
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n): ...
```

Какая структура для какой задачи

Список — порядок важен, дубликаты допустимы, обход по индексу.

Кортеж — неизменяемая запись, можно как ключ словаря.

Словарь — доступ по ключу O(1), справочники, группировки.

Множество — уникальность, операции теории множеств, быстрые проверки.

Проверка «есть ли?» — всегда set/dict, никогда list (если n велико).

Сортировка нужна? — хеш-структуры не хранят порядок: sorted().

Ключ хеш-таблицы — только неизменяемые типы: str, int, tuple.

Справочник скважин

- Дано: список пар «скважина → куст».
- Задача: быстро отвечать «в каком кусте скважина?».
- Решение: dict из пар — O(n) на построение,
- O(1) на каждый запрос.
- Отсутствующие — через get с понятным сообщением.
- Замена повторных linear_search.

код

```
pairs = [('К-12', 'Куст 1'), ('К-13', 'Куст 2'),
         ('К-14', 'Куст 1')]
catalog = dict(pairs)

def cluster_of(wid):
    return catalog.get(wid, 'нет в реестре')

print(cluster_of('К-14'))   # Куст 1
print(cluster_of('К-99'))   # нет в реестре
```

Сводка по кустам

- Дано: пары (куст, дебит).
- Задача: средний дебит по каждому кусту.
- defaultdict(list) собирает, словарное включение считает.
- Это ручной GROUP BY за один проход O(n).
- Пустые группы не появятся — defaultdict сам создаёт.
- Проверяйте деление на ноль — группы непустые по построению.

код

```
from collections import defaultdict

rows = [('Куст 1', 85.4), ('Куст 2', 12.7),
        ('Куст 1', 65.3), ('Куст 2', 40.0)]

groups = defaultdict(list)
for cluster, q in rows:
    groups[cluster].append(q)

print({c: round(sum(v)/len(v), 2)
       for c, v in groups.items()})
# {'Куст 1': 75.35, 'Куст 2': 26.35}
```

Дубликаты в реестре

- Задача: найти повторяющиеся идентификаторы.
- set хранит виденное, дубликат — если уже был.
- Один проход O(n), без вложенных циклов.
- Наивное сравнение всех со всеми — O(n²).
- Отчёт: список дубликатов и их количество.
- Частая задача при проверке качества данных.

код

```
ids = ['K-12', 'K-13', 'K-12', 'K-14', 'K-13']

seen = set()
dups = []
for x in ids:
    if x in seen:
        dups.append(x)
    seen.add(x)

print(dups)      # ['K-12', 'K-13']
```

Сравнение наборов: новые и ушедшие

- Дано: скважины за прошлый и этот месяц.
- Кто появился? Кто исчез? Кто остался?
- Все три ответа — операции множеств: -, &, ^.
- O(n + m) вместо двух вложенных циклов.
- Классическая задача мониторинга и сверки списков.
- Результат готов для отчёта по каждому случаю.

код

```
prev = {'K-12', 'K-13', 'K-14'}
curr = {'K-13', 'K-14', 'K-15'}

print('новые:   ', curr - prev)   # {'K-15'}
print('ушедшие: ', prev - curr)   # {'K-12'}
print('остались:', prev & curr)   # {'K-13','K-
14'}
print('изменился состав:', prev ^ curr)
# {'K-12', 'K-15'}
```

Валидация допустимых значений

- Есть список допустимых статусов/тегов/категорий.
- Каждая входная запись проверяется на допустимость.
- Множество допустимых + проверка in за O(1).
- Неизвестные значения собираем в отдельный список.
- Набор полей записи — set, сравниваем с требуемым (<=).
- Типовая проверка перед загрузкой в базу.

код

```
allowed = {'ok', 'warn', 'err'}
records = ['ok', 'err', 'ok!', 'warn']

unknown = [r for r in records if r not in
allowed]
print(unknown)        # ['ok!']

need = {'дебит', 'W'}
row = {'дебит': 85.4}
print(need <= row.keys())   # False
```

Маппинг ID → объект

- Записи лежат списком, обращаться удобнее по ID.
- Построим словарь: id -> запись (целиком).
- Дальше — произвольный доступ O(1) по любому ID.
- Обновление записи: db[id]['поле'] = новое.
- Проверка на дубликаты ID при построении.
- Это типовая подготовка данных перед анализом.

код

```
records = [
    {'id': 'K-12', 'q': 85.4},
    {'id': 'K-13', 'q': 12.7},
]
db = {r['id']: r for r in records}

db['K-12']['q'] = 90.0      # обновление
print(db['K-13']['q'])      # 12.7
print(len(db))              # 2
```

Изменяемые ключи

- Ключ должен быть хешируемым: str, int, float, tuple.
- Список, словарь, множество — НЕЛЬЗЯ: unhashable.
- Причина: изменение ключа меняет хеш —
- элемент «потеряется» в таблице.
- Кортеж с числами внутри — можно.
- Кортеж со списком внутри — нельзя (внутри изменяемое).

консоль Python

```
>>> d = {(2024, 1): 85.4}    # OK
>>> d[[1, 2]] = 5
TypeError: unhashable type: 'list'
>>> s = {{1, 2}: 'x'}
TypeError: unhashable type: 'set'

>>> d[(1, [2])] = 5          # кортеж со списком
TypeError: unhashable type: 'list'
```

Порядок и хеш-рандомизация

- dict: порядок ВСТАВКИ гарантирован с Python 3.7.
- set: порядок НЕ гарантирован вообще.
- Не пишите код, зависящий от порядка множества.
- hash(str) различается между запусками (защита).
- Поэтому «порядок вывода set» может меняться.
- Нужен порядок — sorted(set) или dict.

правила

```
>>> {'b', 'a', 'c'}    # порядок не важен

# НЕЛЬЗЯ полагаться на:
first = list(s)[0]     # какой элемент?

# МОЖНО:
first = sorted(s)[0]
ordered = list(dict.fromkeys(events))

# dict гарантирует вставку:
list(d.keys()) == порядок добавления
```

Когда dict/set не нужен

- Маленькие данные (десятки) — list проще и быстрее на практике.
- Нужен порядок и дубликаты — только list/tuple.
- Нужен доступ по ИНДЕКСУ (позиции) — list.
- Хеш-структуры тратят больше памяти.
- Итерация по dict/set медленнее, чем по list.
- Выбор структуры — компромисс, а не догма.

```
гайд
# нужен 2-й элемент? -> list
row[1]                    # O(1)

# нужны дубликаты? -> list
log = ['ok', 'ok', 'err']

# нужен порядок следования? -> list/dict
timeline = [('08:00', 85.4), ('09:00', 84.0)]

# нужен быстрый 'есть ли?' -> set/dict
known = set(log)
```

Типичные ошибки и ловушки

{} — это словарь!	пустое множество — только set().
d[k] на отсутствии	KeyError; безопасно — d.get(k, default).
in d.values()	поиск по значению O(n); по ключам — O(1).
Изменяемые ключи	list/set/dict нельзя; tuple — можно.
Удаление во время перебора	собирайте изменения отдельно.
set вместо порядка	set не упорядочен; нужен порядок — sorted/dict.
2 и 2.0 и True	в множестве это один элемент (== равенство).

Итоги лекции

1. Хеш-таблица: ключ -> хеш -> корзина; поиск/вставка/удаление — O(1) в среднем.
2. dict: get/pop/update/keys/values/items; KeyError лечится get(key, default).
3. set: уникальность, | & - ^, подмножества; frozenset — неизменяемый вариант.
4. in по set/dict — O(1); по list — O(n): частые проверки = set/dict.
5. Counter и defaultdict заменяют ручные счётчики и группировки.
6. Ключи — только неизменяемые типы; {} — словарь, set() — множество.
7. Дедупликация с порядком: list(dict.fromkeys(seq)).

Вопрос 1 из 6

Почему доступ по ключу в словаре O(1)? Что происходит при коллизии?

Вопрос 2 из 6

Чем d.get(key) отличается от d[key] и когда что использовать?

Вопрос 3 из 6

Почему ключи словаря и элементы множества должны быть неизменяемыми?

Вопрос 4 из 6

Чем set отличается от list и когда set даёт выигрыш?

Вопрос 5 из 6

Что вернут A - B и A & B для множеств? Приведите пример из аналитики.

Вопрос 6 из 6

Как убрать дубликаты из списка, сохранив порядок элементов?